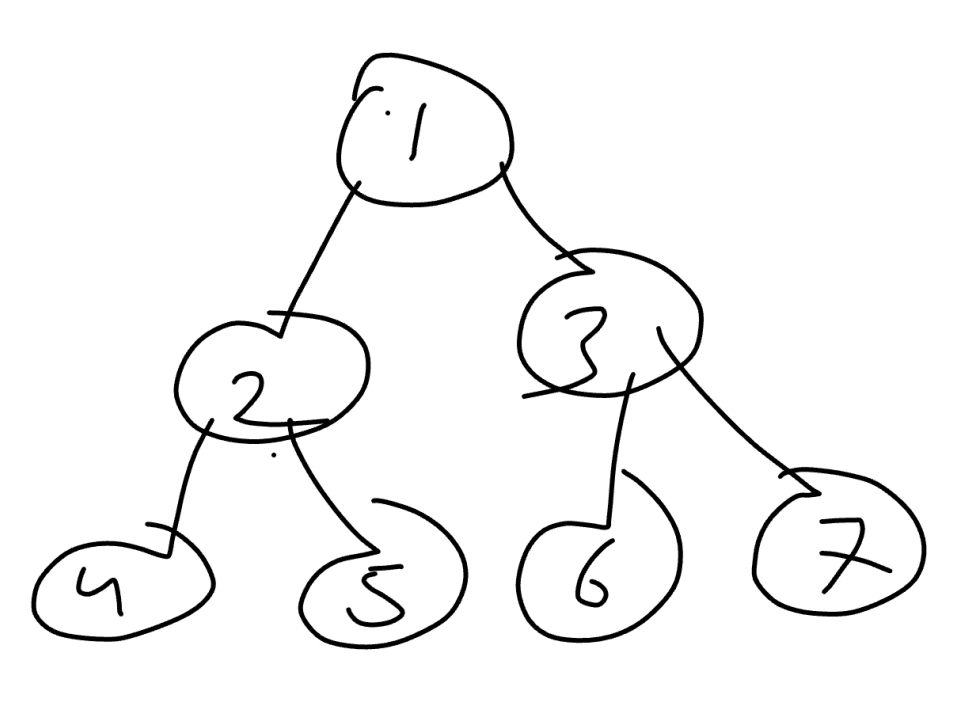
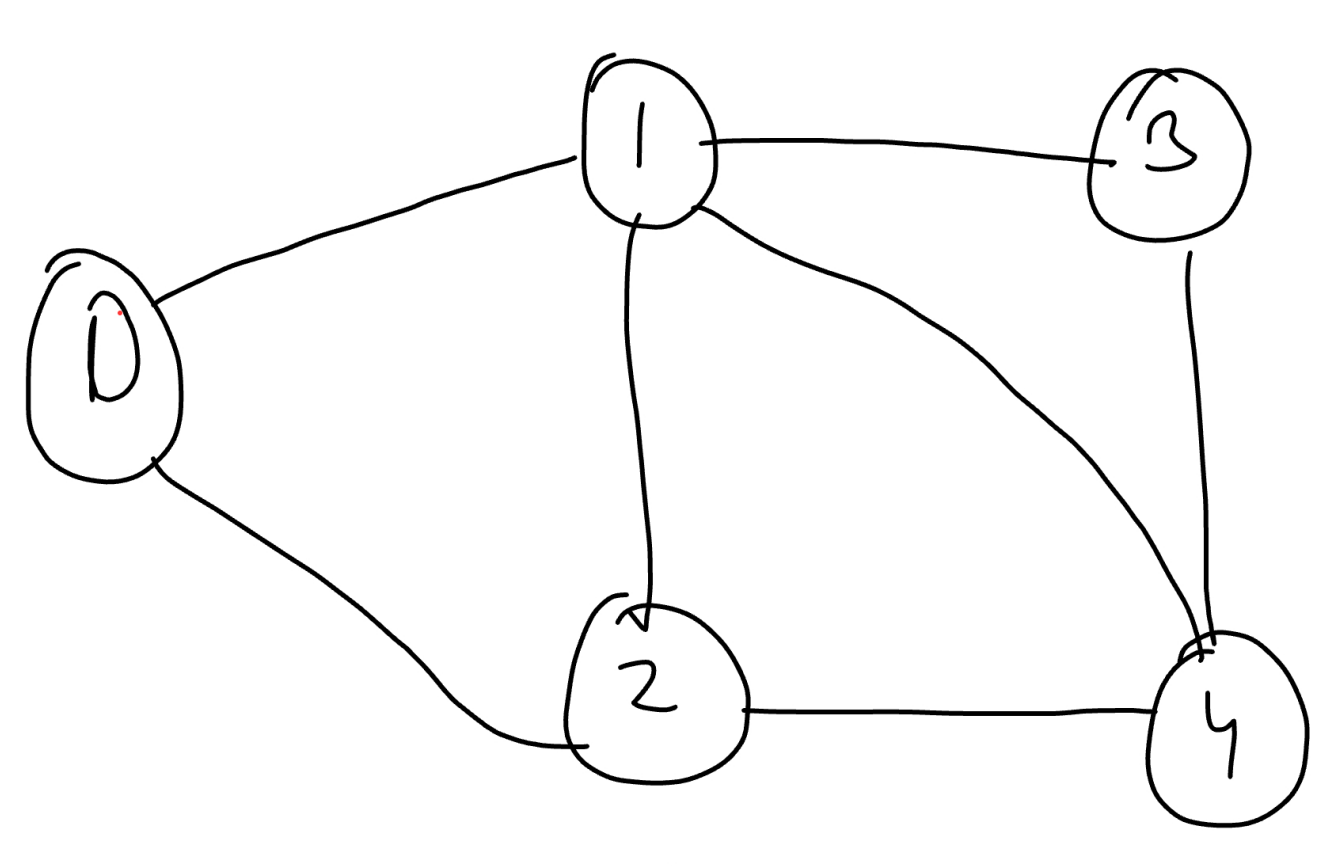
DFS.

Preorder Postorder Inorder.

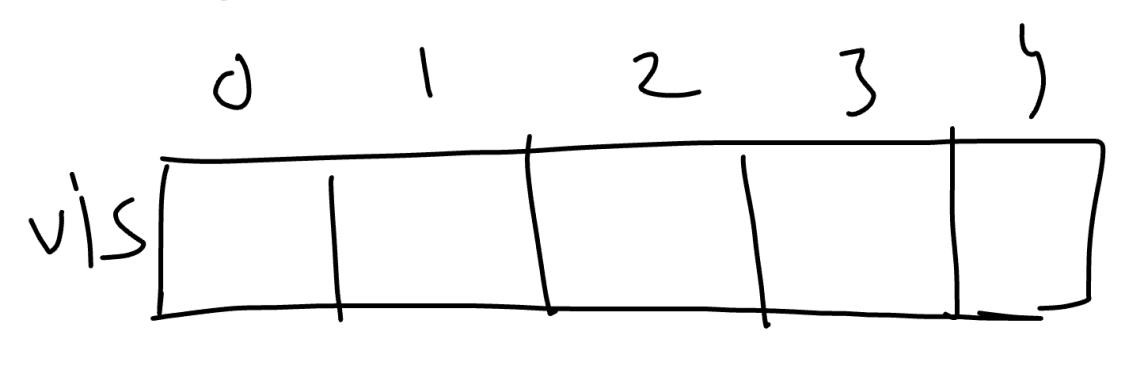
BFS.



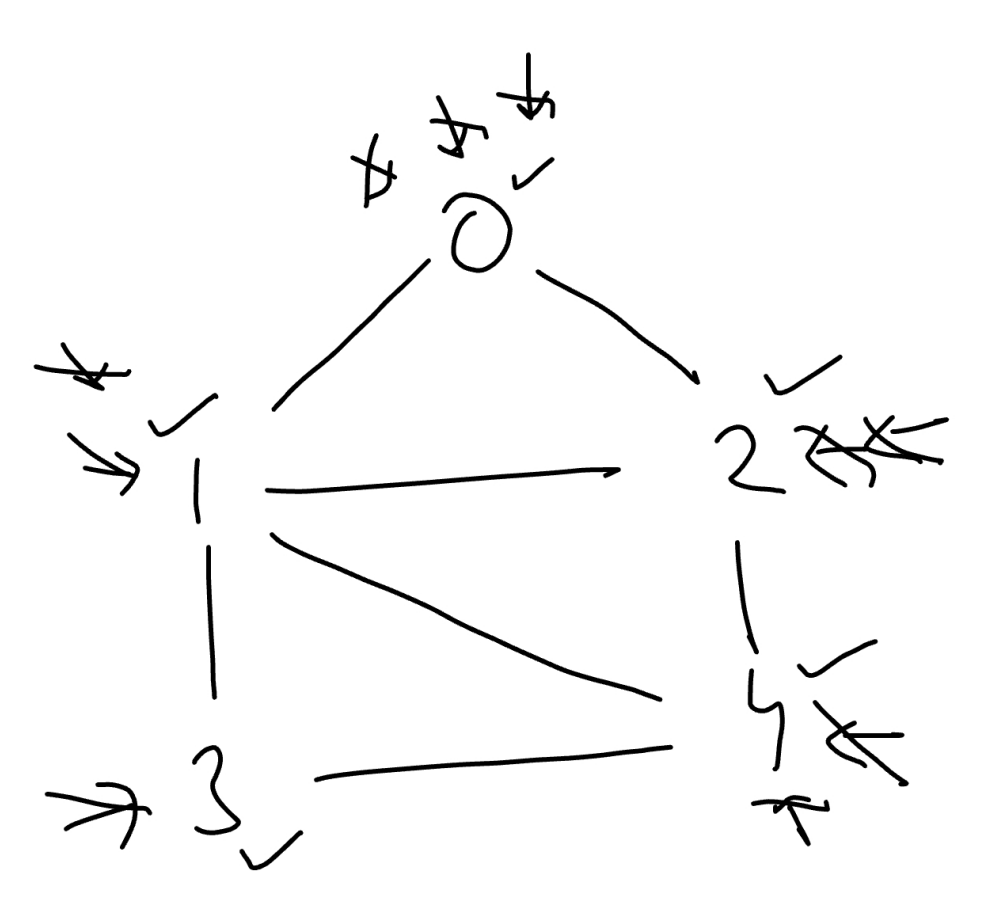


0 → 1, 2
1 → "0, 2, 3, 4"
2 → 0, 1, 4
3 → 1, 4
4 → 1, 2, 3

visited Mapping.

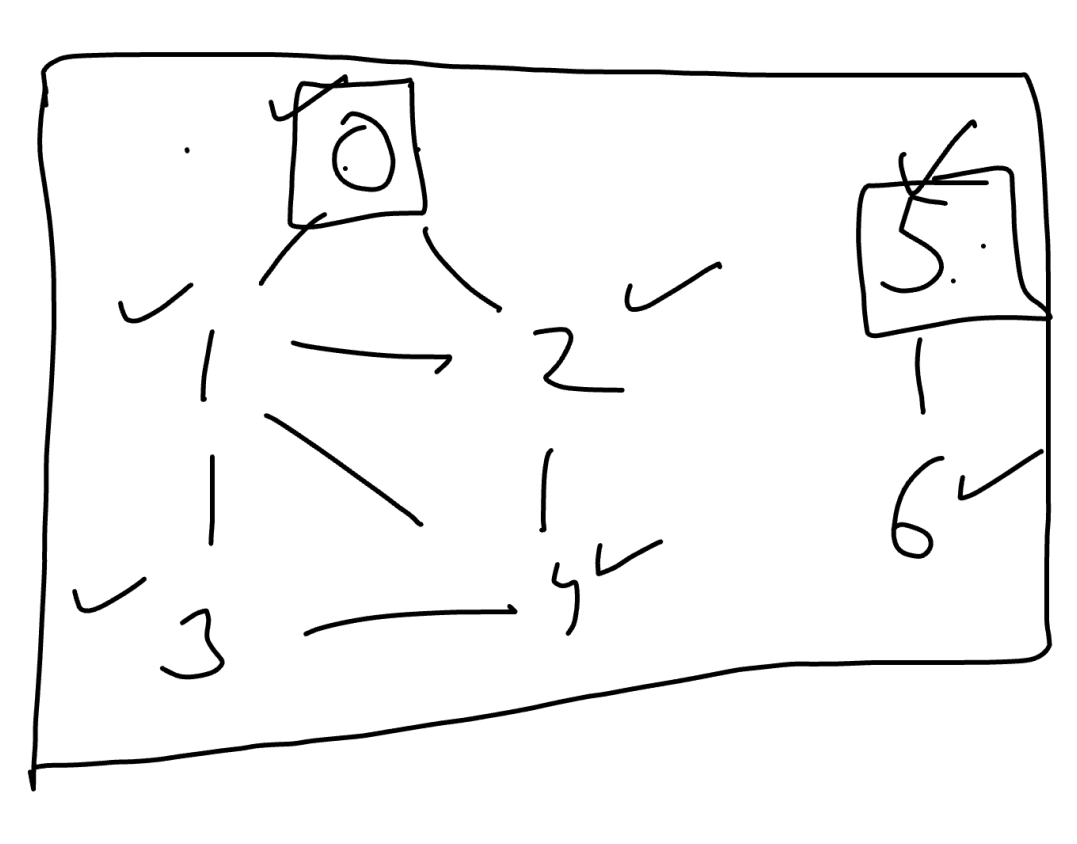


V=5.



O/P: 0, 1, 2, 4, 3

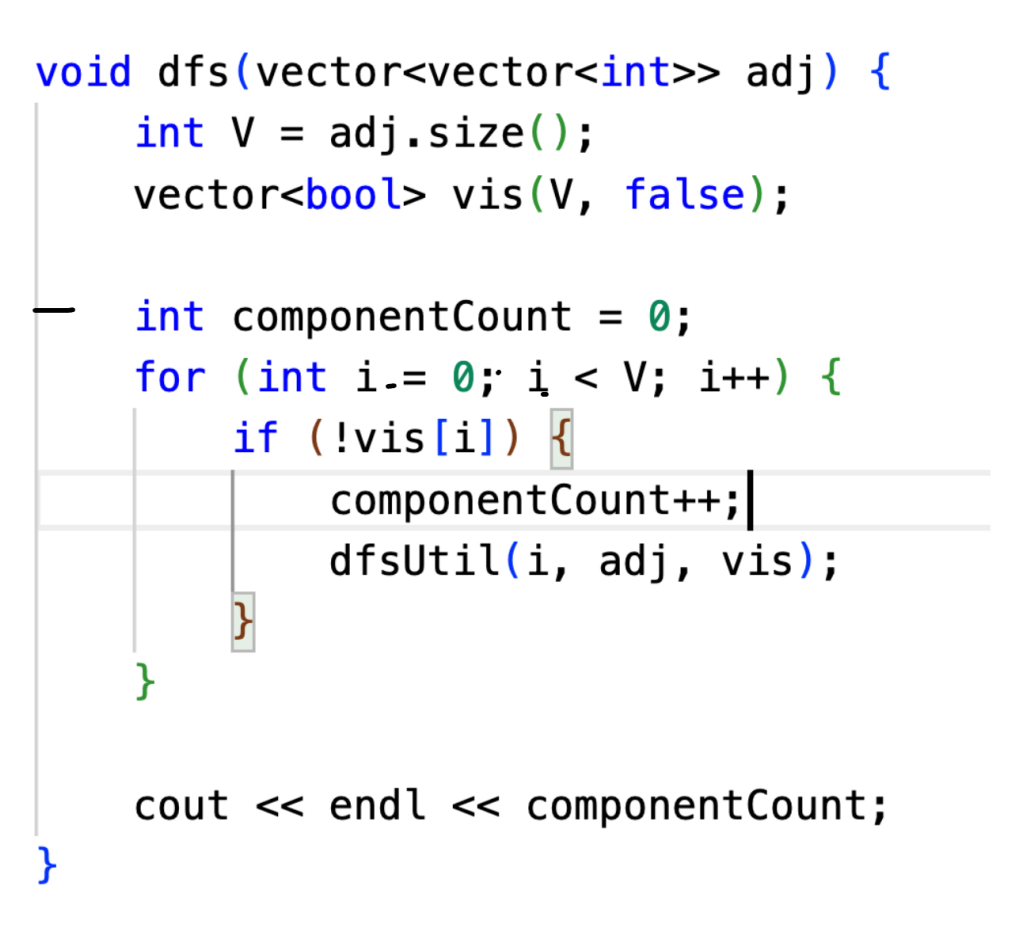
NOTE: For a graph, multiple valid DFS outputs exist.



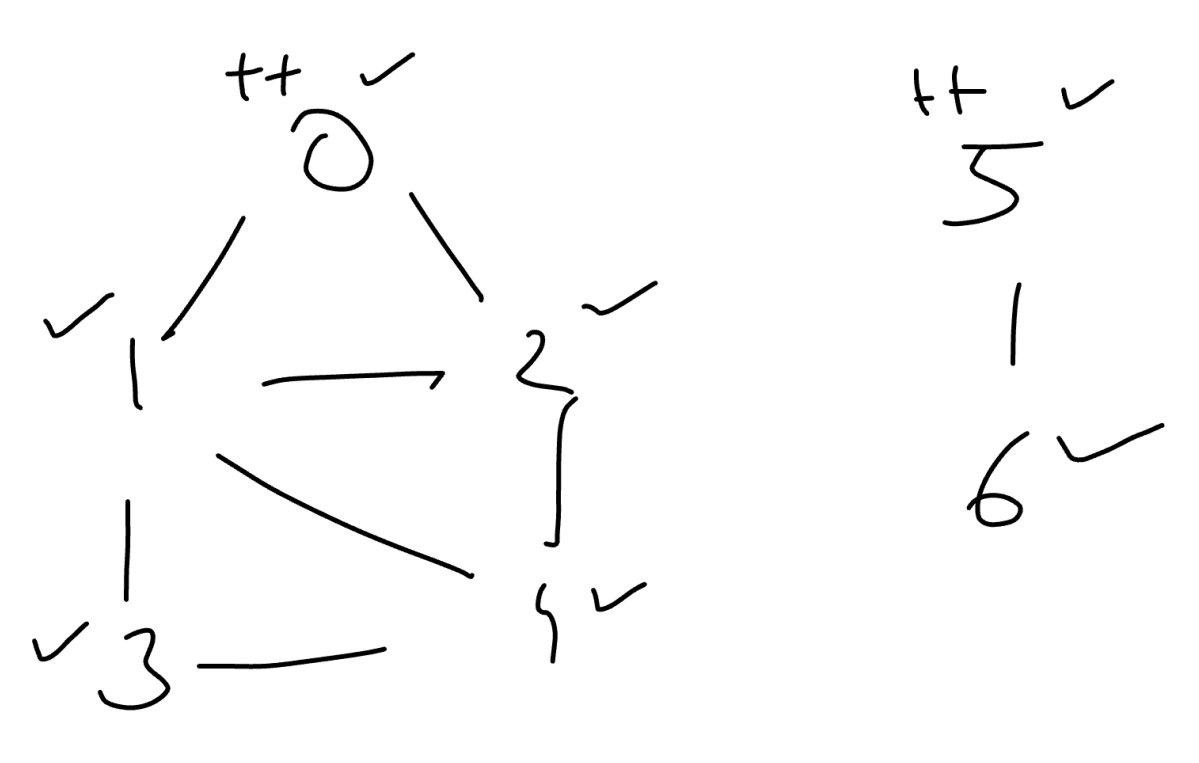
0 → 1, 2
1 → "0, 2, 3, 4"
2 → 0, 1, 4
3 → 1, 4
4 → 1, 2, 3
5 → 6
6 → 5

0, 1, 2, 4, 3, 5, 6

O(E).

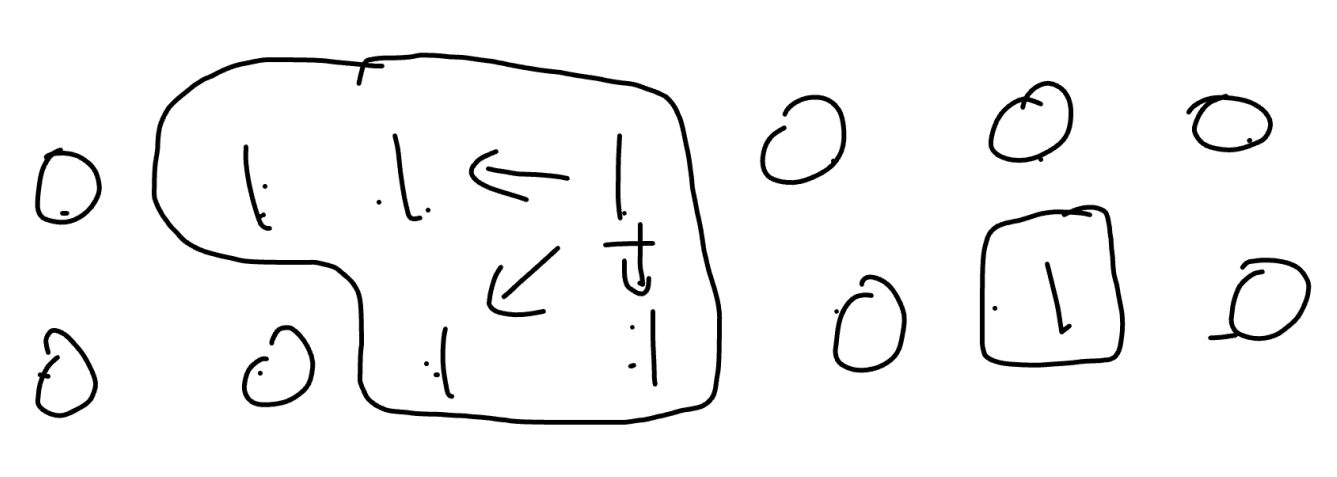


```
void dfs(vector<vector<int>> adj) {
    int V = adj.size();
    vector<bool> vis(V, false);

    int componentCount = 0;
    for (int i.= 0; i < V; i++) {
        if (!vis[i]) {
            componentCount++;
            dfsUtil(i, adj, vis);
        }
    }

    cout << endl << componentCount;
}
```

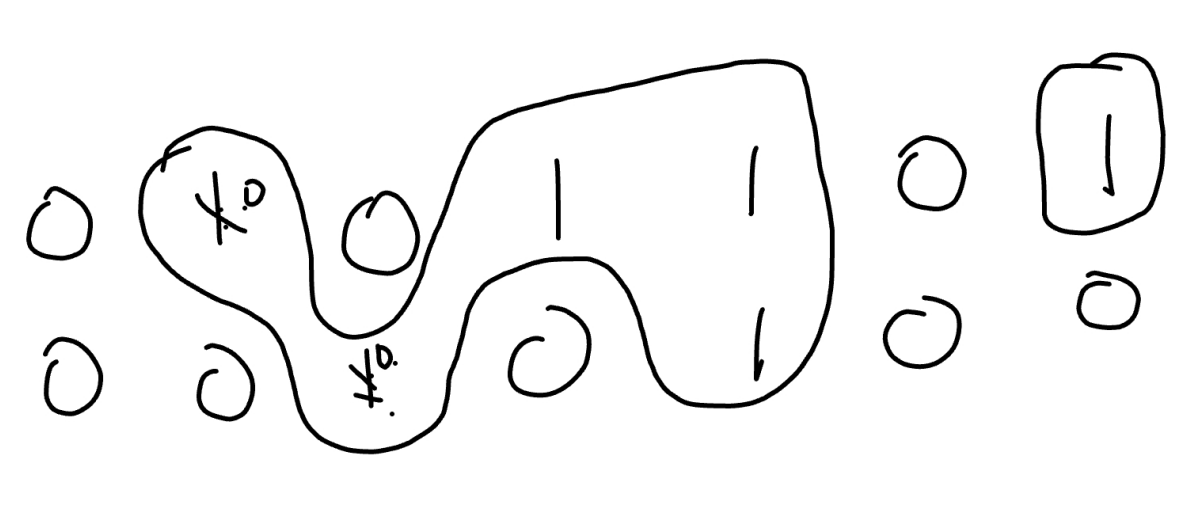


CC = ~~0~~
~~1~~
2

i = ~~0~~
~~1~~
~~2~~
~~3~~
~~4~~
~~5~~ ~~6~~ 7


0 1 1 1 0 0 0
0 0 1 1 0 1 0

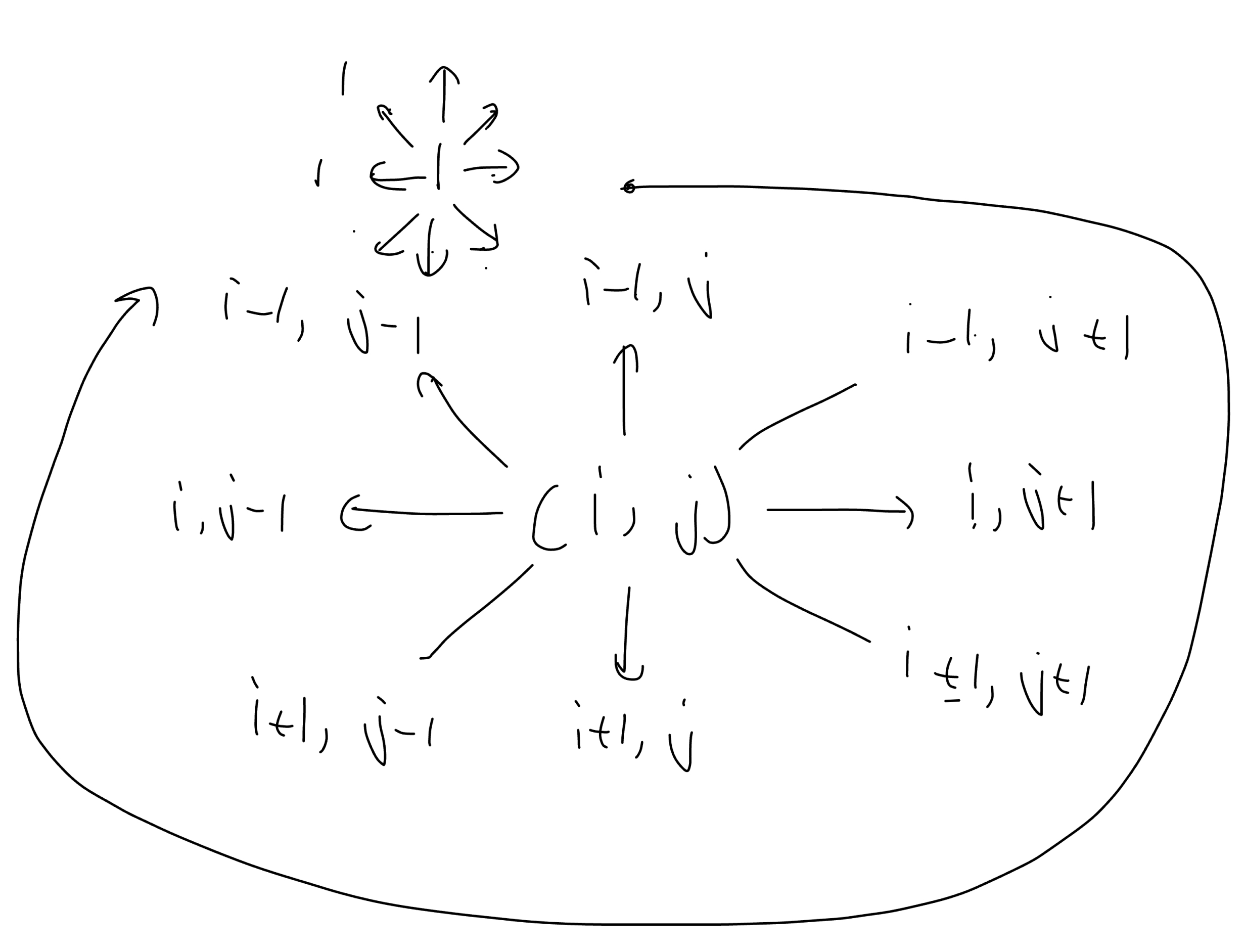
2.


0 1 0 1 1 0 1
0 0 1 0 1 0 0

2.

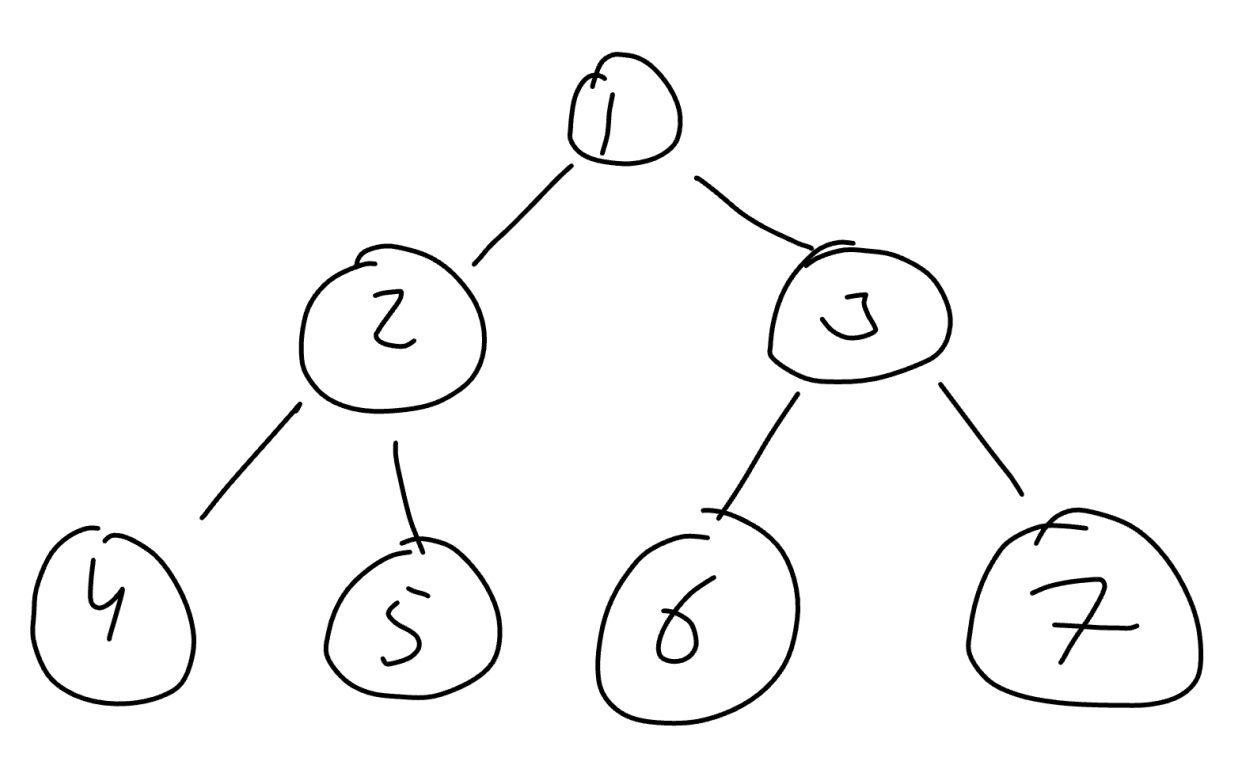
Graph is a list of nodes connected via edges.

Consider all 1s in the grid as your nodes.


1
i-1, j-1
i-1, j
i-1, j+1
i, j-1
(i, j)
i, j+1
i+1, j-1
i+1, j
i+1, j+1

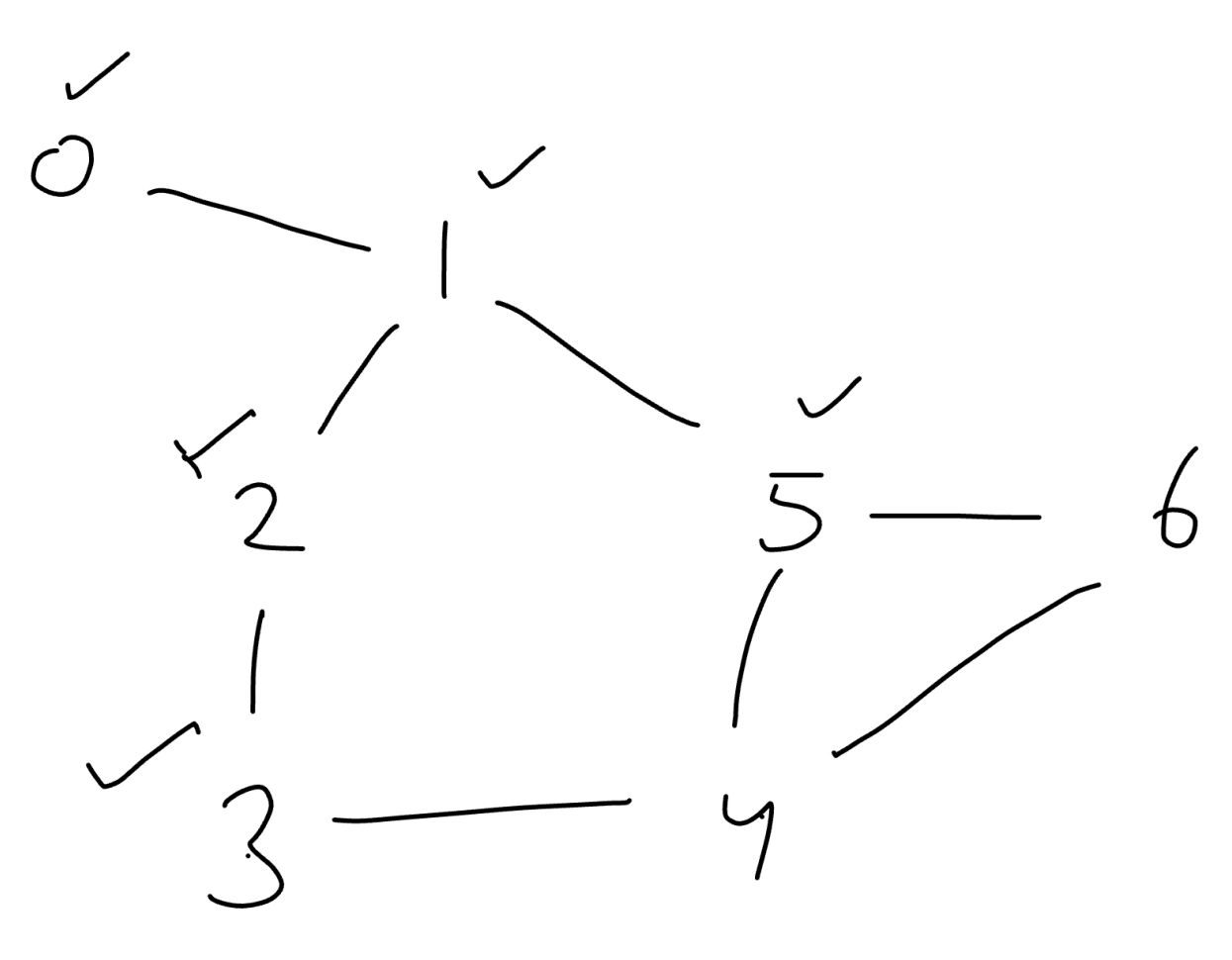
BFS. Traversal

Level-order traversal



1
2 3
4 5 6 7

⇒ Maintain a vis_mapping for graphs



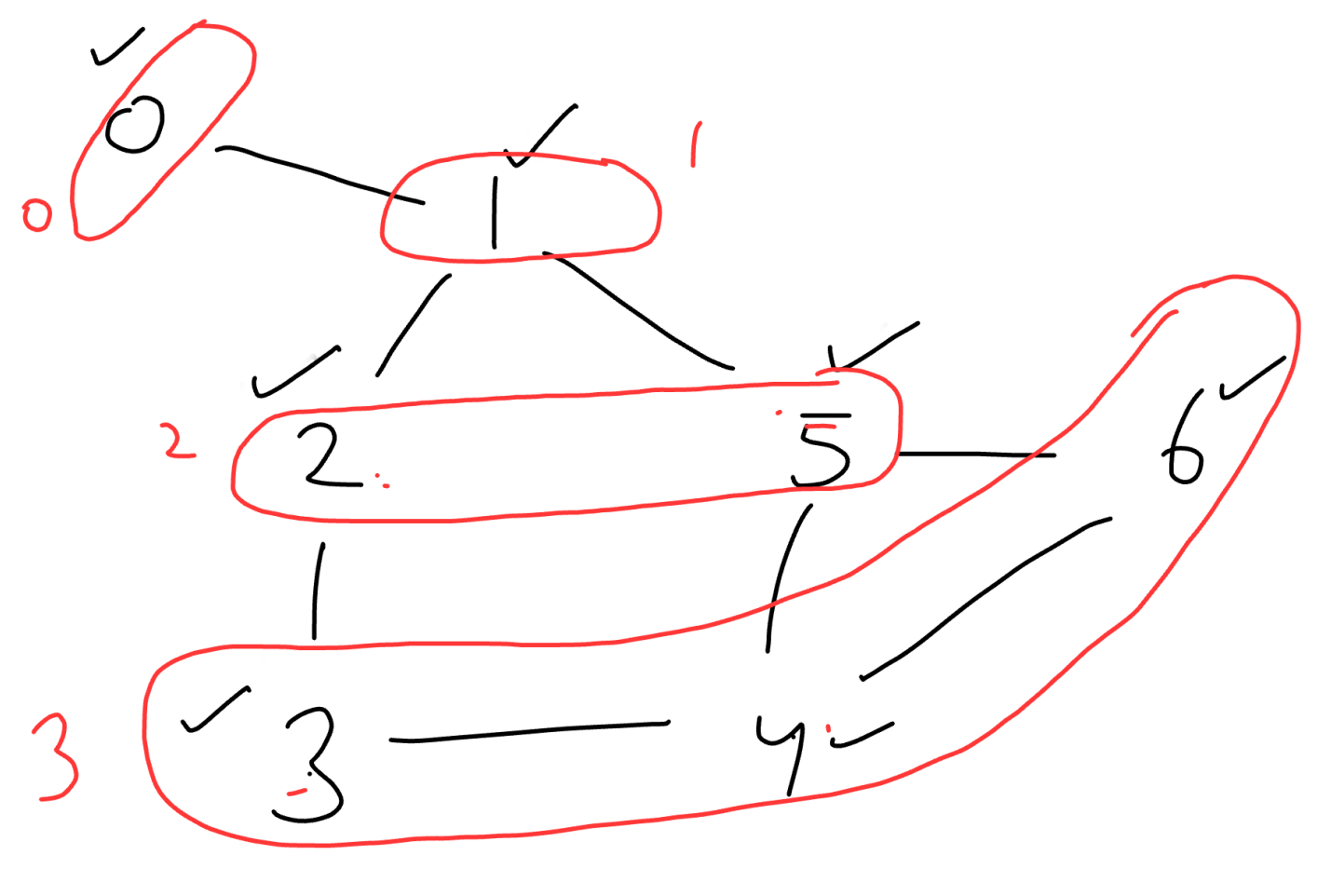
0 → 1
1 → 0, 2, 5
2 → 1, 3
3 → 2, 4
4 → 3, 5, 6
5 → 4, 6
6 → 4, 5

q: ~~0~~ ~~1~~ ~~2~~ ~~5~~ ~~3~~. 4, 6, 4

0, 1, 2, 5, 3

NOTE → Instead of marking the node as visited at the time of q-pop, we will mark it visited at q-push.

7–8


0
1
2
3
0
1
2
5
6
3
4

0 → 1
1 → 0, 2, 5
2 → 1, 3
3 → 2, 4
4 → 3, 5, 6
5 → 4, 6
6 → 4, 5
7 → 8
8 → 7

E

q: ~~0~~ ~~1~~ ~~2~~ ~~5~~, ~~3~~, ~~4~~, ~~6~~

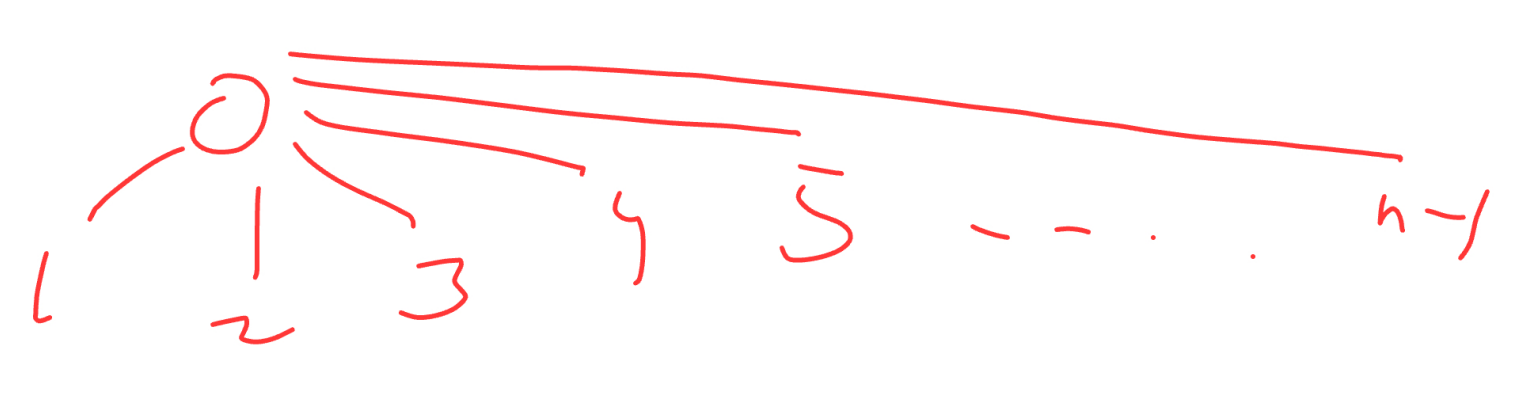
0, 1, 2, 5, 3, 4, 6 O/P.

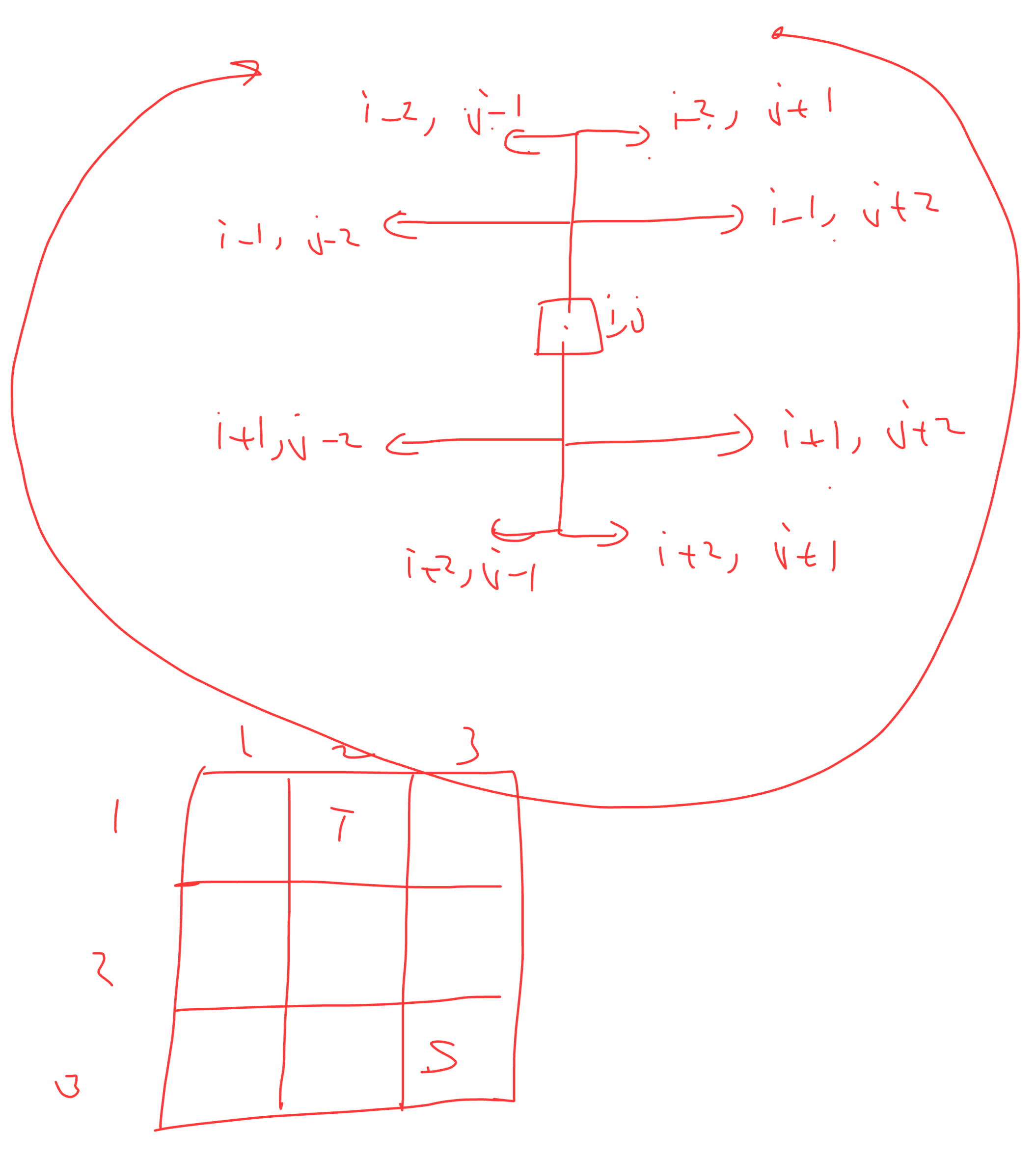
Single-source shortest path can be calculated using a simple level-by-level BFS starting at the source node.

BFS

SSSP algo.

single source shortest path.




i-2, j-1
i-2, j+1
i-1, j-2
i-1, j+2
i, j
i+1, j-2
i+1, j+2
i+2, j-1
i+2, j+1
1
2
3
1
2
3
T
S

Horse.